

Chapter #4:

Informed Search:-

⇒ Informed search, also known as heuristic search, is a problem-solving approach that utilizes domain specific knowledge to make intelligent decisions during the search process.

⇒ Informed search algorithm contains an array of knowledge about search space, such as how far we are from the goal, path cost, how to reach goal node.

⇒ This knowledge help agents to explore less to the search space and find more efficiency the goal node.

⇒ It is useful for large search space. Informed search algorithm uses the idea of heuristic, so it is also called heuristic search.

Components of informed search:

1. State space:

⇒ Informed search algorithms operate within a state space, which represents all possible states that a problem can go through from the initial state to a goal state.

⇒ States are connected by operators or actions that can be applied to transition from one state to another.

2. Heuristics:

⇒ Heuristics are domain specific rules or functions that estimate the cost or distance from a given state to the goal state. Heuristics are used to prioritize which states to explore next.

⇒ It takes the current state of the agent as its input and produces the estimation of how close agent is from the goal.

⇒ The heuristic method, however, might not always give

the best solution, but it guaranteed to find a good solution in reasonable time.

⇒ it is represented by $h(n)$ and it calculates the cost of an optimal path between the pair of states. The value of the heuristic function is always positive.

⇒ Admissibility of heuristic function is given as:

$$h(n) \leq h^*(n)$$

↙ heuristic cost ↘ estimated cost

3. Search Strategy:

⇒ Informed search algorithm combine heuristics with a search strategy to explore the state space efficiently. Common search strategies include A* search, greedy search, and best first search.

Importance of informed search:

Efficiency:

Typically explore several states.

Optimality:

When using an admissible heuristic, informed search algorithms like A* guarantee finding the optimal solution.

Real world applications:

Like:

- Route planning
- Robotics
- Game playing
- Natural language processing

Complex problem solving:-

It enables AI systems to tackle problems with large state space that would be impractical to solve using brute force methods.

Limitations and challenges:

⇒ Informed search is powerful, but it has limitations.

1. Heuristic Quality

◦ The quality of the heuristic greatly influences the effectiveness of informed search.

2. Memory and time Complexity:

◦ Memory and time complexity requirements can be substantial in some cases, especially in large state space.

Applications of Informed Search:

Informed search has a wide range of applications in AI including:

1. Path-finding:

◦ finding optimal routes in transportation, logistics, and map navigation.

2. Game Playing:

◦ Such as video, chess and Go games.

3. Robotics:

◦ Helps robots plan efficient paths and actions to accomplish tasks.

4. Natural language processing:

◦ Plays a crucial role in natural language understanding and machine translation.

5. Planning & Scheduling:

◦ In domains like manufacturing and project management, informed search aids in planning and scheduling tasks.

1. Greedy Best First Search

- It is an informed search algorithm used to find a path from a starting state to a goal state in a graph or search space.

- It is called greedy because it makes decisions solely based on heuristic function that estimates how close a given state is to the goal state. Combination of BFS & DFS.

- Greedy BFS only considers the heuristic estimate ($h(n)$) when selecting the next state to explore.

- It prioritizes nodes that seem to be closest to the goal without considering the actual cost to reach those nodes.

Steps:

1. Initialization:

Create an open list to store nodes that need to be explored.

- Add the starting node to the open list.

2. Main loop:

- While the open list is not empty, repeat the following steps:

3. Select the Node with the lowest Heuristic value:

- Choose the node from the open list that has the lowest heuristic value ($h(n)$). This node is considered the most promising node based solely on the heuristic estimate.

4. Check for the goal state:

If the selected node is the goal state, you have found a path, and the algorithm terminates. You can then backtrack from the goal state to reconstruct the path.

5. Expand the current Node:

- Generate the successor nodes of the current node. These are the nodes that can be reached by applying valid actions from the current node.

- For each successor node:

- (1) Calculate the heuristic value ($h(n)$) for the successor node using a heuristic function.

- (2) Add the successor node to the open list.

6. Loop Termination:

- if the open list becomes empty and the goal state has not been reached, the algorithm terminates, indicating that there is no path to the goal.

7. Path Reconstruction:

if the goal state is reached, you can construct the path

by tracking from the goal node to the start node.

- However, the path found by the greedy algorithm is not guaranteed to be optimal. It may be suboptimal if the heuristic estimate is not accurate.

Pros:

- more speed
- it explores fewer nodes bcz it prioritizes nodes based on heuristic estimate.

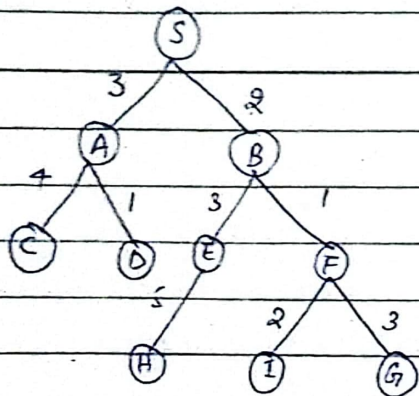
Cons:

- Does not give guarantee to find the optimal path.
- it may find a solution more quickly but may not find the shortest path, especially if the heuristic estimate is not accurate.

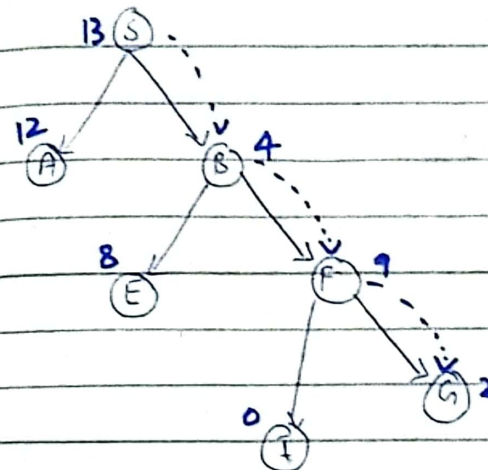
Time and Space Complexity:

$$O(b^m)$$

Example:



node	H(n)
A	12
B	4
C	7
D	3
E	8
F	2
H	4
I	9
S	13
G	0



⇒ Expands the nodes up to S and put in the closed list

⇒ Two lists are used:

- Open
- Closed

Initialization: Open [A], Closed [S, B]

Iteration 1: Open [A], Closed [S, B]

Iteration 2:

Open [E, F, A], Closed [S, B]

Open [E, A], Closed [S, B, F]

Iteration 3:

Open [I, G, E, A], Closed [S, B, F]

Open [I, E, A], Closed [S, B, F, G]

final solution:

I → B → F → G

Completeness: NO

Optimality: NO

Time Complexity: $O(b^m)$

Space Complexity: $O(b^m)$

Algorithm:

Priority Queue Containing initial states

Loop

if PQ - Empty Return Fail

Else

NODE ← Remove-First(PQ)

if NODE = GOAL

Return path from initial
to NODE

Else

Generate all successors of
NODE into PQ according to the cost
value

END LOOP

2. A* Search Algorithm:

⇒ The A* algorithm is a popular informed search algorithm used for finding the shortest path from a starting state to a goal state in a graph or grid.

⇒ It uses heuristic function $h(n)$, and cost to reach the node n from the start state $g(n)$.

⇒ It has combined features of UCS and greedy-best-first search, by which it solves the problem efficiently.

⇒ A* algorithm finds the shortest path through the search space using the $h(n)$.

⇒ This algorithm expands less search tree and provides optimal result faster.

⇒ A* algorithm is same like UCS except that it uses $g(n) + h(n)$ instead of $g(n)$.

⇒ In A* search algorithm, search heuristic as well as the cost to reach the node.

⇒ Both costs are combined and this sum is called fitness number.

$$f(n) = g(n) + h(n)$$

Estimated Cost of the Cheapest solution Cost to reach node n from start state Cost to reach from node n to goal node

⇒ At each point in the search space, only those node is expanded which have the lowest value of $f(n)$, and the algorithm terminates when the goal node is found.

⇒ How the algorithm work steps are:

1. Initializations

- Create an open list to store nodes that need to be explored.
- Add the starting node to the open list with an initial 'g' value of 0.

○ Initialize the 'g' values for all other nodes as infinity and the 'h' values (sum of 'g' and 'h') as infinity as well.

2. Main loops

- While the open list is not empty, repeat the following steps:

3. Select the node with the lowest f value

- Choose the node with the lowest 'g' value from the open list. This node will be the one to be expanded next.

4. Check for the goal state:

- If the selected node is the goal state, you have found the shortest path, and the algorithm terminates.

from here on -
expanded, expanded, expanded

- Then backtrack from the goal state to reconstruct the path.

5. Expand the current Node:

- Generate the successor nodes of the current node n and put n into the closed list.
- For each successor n , check whether n is already in the OPEN or CLOSED list, if not then compute evaluation function $f(n)$ and place into Open list.

6.

Else if node ' n ' is already in OPEN and CLOSED then it should be attach to the back pointer which reflects the lowest $g(n)$ value.

6. Loop termination:

- If the open list becomes empty, and the goal state has

not been reached, the algorithm terminates, indicating that there is no path to the goal.

7. Path Reconstruction:

If the goal state is reached, the reconstruct the shortest path by backtracking from the goal node to the start node using the parent pointers set during the search.

Note: The key to the efficiency and optimality of A^* lies in the heuristic function $h(n)$. If $h(n)$ is admissible and consistent (satisfies triangle inequality), A^* is guaranteed to find the optimal solution.

However, the quality of the heuristic greatly influences the algorithm's performance. A better heuristic leads to fewer nodes being expanded, resulting in faster searches.

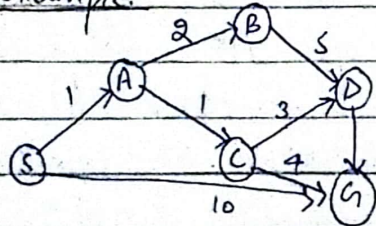
Pros:

- it is Optimal and Complete.
- it can solve complex problem.

Cons:

- Does not always produce the shortest path as it is mostly based on heuristic and approximation.
- Memory requirements to keep all generated nodes. Complexity issues.

Example:



state	$h(n)$
S	5
A	3
B	4
C	2
D	6
G	0

$$S \rightarrow A = 1 + 3 = 4 \checkmark$$

$$S \rightarrow G = 10 + 0 = 10$$

$$S \rightarrow A \rightarrow B = 1 + 2 + 4 = 7$$

$$S \rightarrow A \rightarrow C = 1 + 1 + 2 = 4 \checkmark$$

$$S \rightarrow A \rightarrow C \rightarrow D = 1 + 1 + 3 + 6 = 10$$

$$S \rightarrow A \rightarrow C \rightarrow G = 1 + 1 + 4 + 0 = 6 \checkmark$$

$$\text{Sequence: } S \rightarrow A \rightarrow C \rightarrow G$$

Complete:

A* algorithm is complete as long as:

- Branching factor is finite.
- Cost of every action is fixed.

Optimal: A* is optimal if it follows two conditions:

- Admissible
- Consistency

Time and space complexity:

- $O(b^d)$
- $O(b^d)$

Algorithm:

- Enter starting node in OPEN List.
- if OPEN List is empty return FAIL
- Select the node from OPEN List which has smallest value $(g+h)$.
 - if node = goal, return success
- Expand node 'n' and generate all successors
 - compute $(g+h)$ for each successor node.
- if node 'n' is already in OPEN (closed, attach to backpointer).
(go to (ii))

Local Search Algorithm & Optimization problems:

⇒ The local search algorithm searches only the final state, not the path to get there.

⇒ Local search algorithms operate by searching from a start state to neighbouring states,

⇒ without keeping track of the paths, nor the set of states that have been searched.

⇒ Local search algorithm is a category of optimization algorithms that operate by iteratively exploring the solution space from a single current solution, typically moving to neighbouring solution and making local improvements.

⇒ They are non-systematic in nature.

⇒ They are useful when search space is very large or continuous or with limited memory.

key points:

1. Single Current Node and Local Explorations

- it maintain a single current solution or node throughout the search process.
- They don't explore the entire state space but rather focus on improving the current solution.
- These algorithms typically examine neighbouring solutions by making small changes to the current solution. The neighbours are generated based on problem specific operators or moves.

2. No Path Retention

- Local search algorithms do not retain any path information.
- The primary focus is on improving the quality of the current solution, not how the solution was reached.

Advantages:

◦ Local Memory Usage:

⇒ LSA are memory efficient. typically requires constant amount of memory to store only the current solution.

◦ Suitability for large or Continuous Spaces:

⇒ They are well suited for optimization problems in a large or continuous spaces where systematic algorithms become impractical due to exponential growth of possibilities.

Optimization problems and the Landscape Analogy:

In optimization problems, the analogy of a landscape with "location" and "elevation".

⇒ The "elevation" represents the objective function value as a measure of solution quality.

⇒ if we seek to minimize an objective function, we are searching for the lowest valleys in the landscape aiming to find a global minimum.

⇒ Conversely, if to maximize an objective function, then searching for the highest peaks in the landscape seeking a global maximum.

⇒ Local search algorithms are particularly effective for optimization problems in which finding a global optimum is challenging or computationally expensive. They are more likely to find "good enough" solutions quickly, even if these solutions are not guaranteed to be the absolute best.

Example:

- Hill climbing
- Simulated Annealing
- Genetic Algorithms

Hill Climbing

it is variant of generate and Test method

⇒ Hill climbing is a local search algorithm used in artificial intelligence and optimization to find a solution to a problem by iteratively making small changes to the current solution and selecting the neighbours with the highest fitness or value.

⇒ its named "hill climbing" because it can be seen as climbing a hill to reach the peak of a fitness landscape. it belongs to a family of local search algorithms.

Features of Hill climbing

Following are some main features of Hill algorithm:

- o Greedy Approach:-

Hill-climbing algorithm search moves in the direction which optimizes the cost.

- o No backtracking.

It does not backtrack the search space, as it does not remember the previous states.

- o Generate and Test variants

Hill climbing is the variant of Generate and Test method.

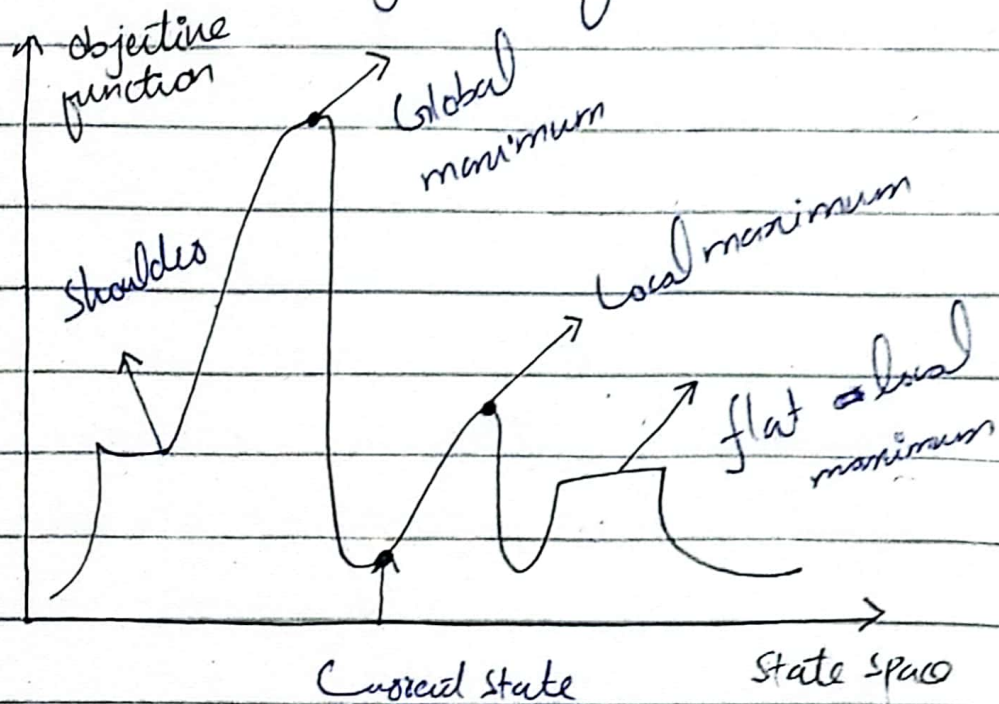
The Generate and Test method produce feedback which helps to decide which direction to move in the search space.

- o Local Search

⇒ it always move in single direction. it allow only up-ward direction. it does not have knowledge of global search space.

State-Space Diagram for Hill Climbing:

⇒ The state-space landscape is a graphical representation of the hill-climbing algorithm which is showing a graph between various states of algorithm and Objective function / Cost.



Different regions in the state space landscape:-

Local maximum:-

⇒ Local maximum is a state which is better than its neighbour states, but there is also another state which is higher than it.

Global Maximum:-

Global maximum is the best possible state of state space landscape. It has the highest value of objective function.

Current state:

It is a state in a landscape diagram where an agent is currently present.

Flat local maximum:

It is a flat space in the landscape where all the neighbour states of current states have the same value.

Shoulders

It is a plateau region which has an uphill edge.

Working of Hill climbing:

1. Initialization:

- Start with an initial solution or state.

2. Evaluation:

- Evaluate the current solution's quality by calculating its fitness or value.

- The fitness function should quantify how good solution is with respect to the problem trying to solve.

Higher values typically represent better solution.

3. Local search:

- Generate neighbouring solutions by making small, incremental changes to the current solution.

For example, in a Traveling Salesman problem, a neighbouring solution could involve swapping two cities in the tour.

4. Selection:

⇒ Select the neighbour with the highest fitness or value as the new current solution.

⇒ If multiple neighbours have the same or similar fitness, then choose randomly or based on a specific strategy.

5. Termination Conditions

- o Repeat step 3 and 4 until a termination condition is met.

This condition could be a maximum number of iterations, a time limit, or a specific fitness threshold.

6. Output

- o Return the current solution as the result, which is expected to be a local optimum.

Types of Hill Climbing Algorithm

- o Simple hill climbing
- o Steepest-Ascent hill-climbing
- o Stochastic hill climbing.

Limitations:

(1) Local optima:

- o Hill climbing is prone to getting stuck in local optima. If the algorithm reaches a peak and there are no better neighbours, it will terminate prematurely, even if a better global optimum exists elsewhere in the search space.

(2) Deterministic:

- o Hill climbing is deterministic; it always chooses the best neighbour if one exists. This can make it less suitable for problems with noise or stochasticity.

Variants:

⇒ Several variants of hill climbing exist to address its limitations such as:

- Random restart hill climbing
- Simulated Annealing
- Genetic Algorithms

Exploration vs Exploitation:

⇒ Hill climbing focuses on exploiting the current best solution. It lacks exploration, which means it might miss out on finding better solutions on other parts of the search space.

Simulated Annealing

Algorithm

⇒ Simulated Annealing is a powerful optimization algorithm inspired by "the annealing process in metallurgy".

⇒ it is a variant of hill climbing and it allows downward steps also.

⇒ it is used to find the global optimum of complex objective functions by gradually cooling a system to escape local optima.

⇒ SA is memory less algorithm. the algorithm does not use any information gathered during the search.

⇒ it is a stochastic global search optimization algorithm.

⇒ it is used for solving unconstrained and bound constrained OP.

Key Concepts:

1. Annealing in metallurgy:

◦ Metallurgical annealing process is defined as:

◦ Heating and cooling a material to remove defects and reach a low energy state.

2. Search space & Objective function

Search space:

◦ The range of possible solutions.

Objective function:

◦ Measures the quality of solutions.

Algorithm:-

Steps that are involved in SA are as follows:

1. Initialization:

◦ Start with an initial solution and set temperature and cooling parameters.

2. Local search:

◦ Generate neighbouring solutions.

3. Evaluation:

◦ Calculate the fitness of the current and neighbouring solutions.

4. Acceptance probability:

◦ Calculate the probability of accepting a neighbouring solution.

5. Accept or reject:

◦ Decide whether to accept the neighbouring solution.

6. Cooling:

◦ Reduce the temperature.

7. Termination:

◦ Define stopping criteria.

Applications:

- ⇒ Traveling Salesman problem
- ⇒ Function Optimization
- ⇒ Parameter tuning in machine learning
- ⇒ protein folding
- ⇒ Scheduling and routing problems

Key point:

⇒ The method models the physical process of heating a material and then slowly lowering the temperature to decrease defects, thus minimizing the system energy.

⇒ This algorithm is appropriate for non-linear objective functions, where local search algorithms do not operate well.

⇒ Very useful for situation where there are a lot of local minima

Genetic Algorithms

it is evolutionary algorithm.

⇒ Genetic algorithms are optimization techniques inspired by the principles of natural selection and genetics.

○ It is an adaptive heuristic search algorithm.

⇒ They belong to a family of stochastic beam search algorithms where successor states are generated probabilistically.

○ it was inspired by "Darwin theory of evolution in Nature"

⇒ GAs differ from traditional search algorithms by introducing the concept of sexual reproduction.

Genetic Algorithms as a stochastic Beam search variant:-

⇒ Stochastic beam search involves exploring a search space probabilistically.

⇒ In GAs, success states are not generated by modifying a single state but by combining information from two parent states.

Key Components of genetic

Algorithms:

Population:

- GA maintains a population of potential solutions. Subset of all possible solutions, which can solve the given problem.

Fitness function:

- An objective function that measures the quality of solutions.

Selection:

- The process of choosing parent states based on their fitness. It is directly proportional to fitness.

Crossover:

- Combining portions of two parent states to create offspring.

Method $\left\{ \begin{array}{l} \rightarrow 1\text{-Point} \\ \rightarrow \text{multi point} \end{array} \right.$

Mutation:

- Introducing a small random changes to a solution. methods: $\left\{ \begin{array}{l} \rightarrow \text{swap} \\ \rightarrow \text{bit-flip} \end{array} \right.$

Termination:

- Conditions for stopping the process. Convergence point where fitness value is very high.

Advantages of Genetic Algorithms:

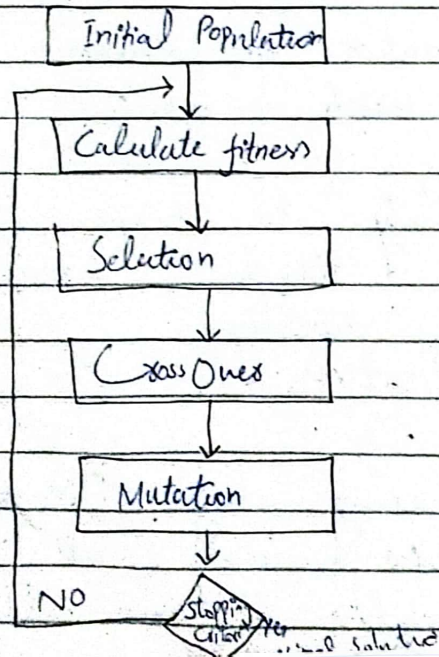
- Suitable for complex optimization problems with rugged search spaces.
- Exploration of diverse regions of the search space.
- Ability to escape local optima through sexual reproduction & genetic diversity.

Applications:

- Engineering design Optimization
- Machine learning
- Scheduling & resource allocation
- Evolutionary robotics

Summary:

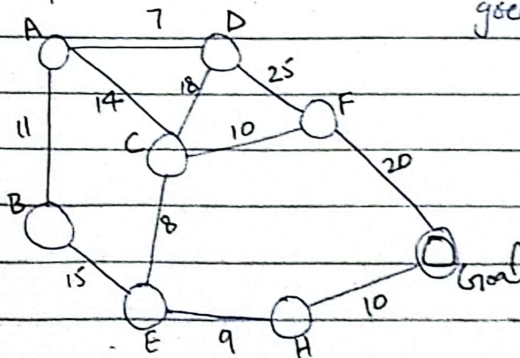
- Genetic Algorithm was introduced by John Holland.
- Abstraction of real biological evolution.
- Solve complex problems (like NP Hard).
- Focus on Optimization.
- Population of possible solutions of a given problem.
- Not efficient for simple problems.



Beam Search Algorithm

- This algorithm basically prunes beam width ^{measure in BFS} beam value is infinite. (constant)
- It takes care of space complexity.
- It is an optimized version of best first search.
- It is heuristic search algorithm.
- Explores a graph by expanding the most promising node in a limited set. Beam width is already given B .
- Reduces memory requirement and time.
- Greedy Algorithm.
- Beam value is already given.
- Predetermined no. of best partial solutions are kept as candidate. Therefore, it is greedy Algorithm.

Example:



Heuristics / straight line dist

A $\rightarrow$ G = 40

B $\rightarrow$ G = 32

C $\rightarrow$ G = 25

D $\rightarrow$ G = 35

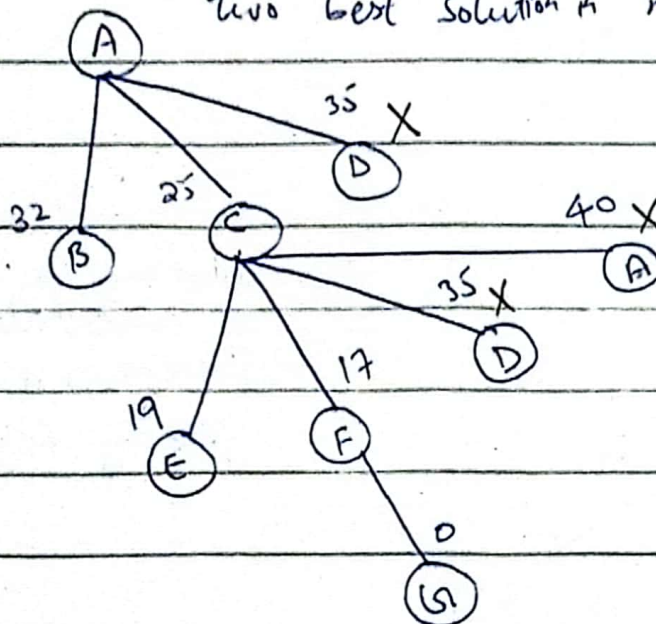
E $\rightarrow$ G = 19

H $\rightarrow$ G = 10

G $\rightarrow$ G = 0

F $\rightarrow$ G = 17

(B=2) — means keep only two best solution in memory.



• A beam search takes uses components:

• A problem to be solved.

• A set of heuristic rules for pruning.

• And a memory with a limited available capacity.

• Not complete

• Not give optimal solution.